AndroNode Whitepaper

Android Distributed AI Compute Protocol

Version 1.0 | 2 November 2026

Abstract

AndroNode is a proposed AI compute protocol that connects compatible Android devices into a shared network for paid inference tasks. Phones, tablets, Android TV boxes, and other tested Android hardware contribute processing capacity under owner-defined limits. Customers submit supported workloads, the network assigns short tasks to eligible devices, and contributors receive payment for accepted results.

The first implementation uses a managed coordinator, a small approved model catalog, trusted result verification, and customer-funded payments. AI execution, scheduling, and verification run offchain. A later paid release uses a single-chain stablecoin escrow for customer funds and contributor withdrawals. A blockchain records settlement; it does not prove that an AI calculation is correct.

This whitepaper defines the proposed operating rules, reference architecture, commercial model, security boundaries, and release criteria. It describes a design baseline, not a claim that an application, audited contract, customer base, token, or compute network is already live. All numerical defaults are proposed engineering or commercial parameters unless explicitly identified as illustrative examples. Changes must be versioned and cannot alter accepted job terms retroactively.

Contents

1. Protocol scope
2. Participants and responsibilities
3. Supported devices and admission
4. Workload catalog
5. Android application and execution
6. System architecture
7. Model registry and runtime profiles
8. Job lifecycle and scheduling
9. Result verification
10. Contributor identity and abuse prevention
11. Customer data and privacy
12. Pricing and contributor payments
13. Settlement and customer funds
14. Native token policy
15. Product interfaces and developer API
16. Security and operational recovery
17. Distribution and operating requirements
18. Pilot and commercial launch plan
19. Governance and protocol changes
20. Risks and limits
21. Release specification
22. Reference job and operating scenarios
23. Technical sources

1 Protocol scope

AndroNode provides asynchronous inference on compatible Android hardware. Each device executes a complete small task independently. The initial network does not combine device memory, split one large model across the internet, train foundation models, or offer guaranteed real-time chat latency.

The first commercial product is a batch embedding service for public or otherwise approved non-sensitive text. Customers use the outputs for semantic search, clustering, and indexing. Text classification can follow after its model and validation rules pass the same release process. OCR and short audio transcription remain later extensions because their payload sizes, model behavior, privacy risks, and verification costs need separate testing.

The network pays for useful accepted outputs. Registration, uptime, screen taps, installing the app, holding a token, or purchasing a membership do not generate base compute payments. There is no fee to become a contributor and no paid multiplier that changes the reward for identical accepted work.

Android is the target device ecosystem, not a promise of universal compatibility. Every supported form factor must pass its own admission and sustained-load tests. Vehicle systems, medical devices, safety-critical controllers, wearables, and shared devices without administrator authorization are outside the initial scope.

The initial protocol is coordinator-managed. Customers trust the operator to quote honestly, assign work, run verification, adjudicate disputes, and authorize settlement. Contributors trust the same operator to accept valid outputs and record payments correctly. Published receipts and onchain balances make parts of this process auditable; they do not remove those dependencies.

2 Participants and responsibilities

Participant	Responsibility	Economic role
Customer	Supplies lawful inputs, selects a model profile, funds an order, retrieves results	Pays for accepted customer records
Contributor	Authorizes a device, sets limits, accepts offered tasks, returns signed results	Earns the quoted amount for accepted work
Coordinator	Quotes orders, issues leases, tracks progress, enforces budgets	Operates the marketplace
Verifier	Recomputes results under a published validation profile and records decisions	Paid from the service charge
Settlement administrators	Approve reconciled settlements through separate signing keys	Control a disclosed trust boundary
Model maintainer	Packages models, licenses, tokenizers, and test vectors	Maintains compatible execution profiles

A contributor account may control multiple devices. Device reputation is separate from account reputation, and tasks are assigned by measured capability and reliability. A TV box is not rewarded more merely because it is always plugged in; it earns only when it completes eligible paid work.

The pilot verifier and coordinator may be operated by the same organization. They use separate services, credentials, and logs, but this is operational separation, not independent governance. External verifiers are a later milestone requiring published admission rules and conflict-of-interest controls.

3 Supported devices and admission

3.1 Initial compatibility requirements

The proposed baseline is Android 11 or later, a 64-bit ARM application environment, at least 4 GB installed RAM, at least 1.5 GB available memory at admission, and 1 GB free storage. A model profile may impose stricter requirements. These are AndroNode launch parameters, not universal requirements imposed by Android or an inference runtime.

The initial paid device list contains only tested model and operating-system combinations. A device that meets the headline specifications can still fail because of unstable drivers, inadequate cooling, manufacturer process restrictions, incorrect system reporting, or incompatible model operators.

Device category	Initial treatment	Additional gate
Android phones	Closed pilot, then approved models	Charging behavior, thermal response, interruption tests
Android tablets	Closed pilot, then approved models	Memory pressure and sustained performance
Android TV and TV boxes	Separate expansion cohort	Cooling, sleep behavior, remote-friendly controls, certification
Other Android hardware	Case-by-case admission	Ownership, runtime compatibility, power and thermal validation
Non-GMS or uncertified devices	Research cohort first	Separate trust and distribution path; no assumed Play Integrity coverage

3.2 Registration and qualification

The owner creates an account, accepts resource-use terms, registers a device signing key, and runs a qualification test. The test measures model loading, output compatibility, memory use, network transfer, and repeated inference under a controlled session. It records both cold-start and sustained performance; a short benchmark is not sufficient evidence of safe overnight operation.

Admission requires at least 1,000 calibration records with no unexplained validation failures, a 30-minute supervised thermal test, and successful stop, disconnection, and process-restart tests. A failure blocks paid work for that profile until reviewed. A software update, runtime change, or material performance regression can require requalification.

Node identity does not depend on collecting an IMEI or an advertising identifier. The device public key is the primary registration handle. Reinstallations may create a new key and must pass probation again. The protocol does not claim one registration equals one physical device or one human owner.

4 Workload catalog

4.1 First workload

The reference profile is short English text embedding with a fixed encoder, tokenizer, pooling procedure, and vector normalization. A candidate model is sentence-transformers/all-MiniLM-L6-v2, whose publisher describes 384-dimensional sentence embeddings and lists an Apache 2.0 license [1]. Its inclusion is a candidate selection, not a claim that an AndroNode mobile build has already passed testing.

The first implementation uses a pinned ONNX Runtime Android CPU build with FP32 inference. ONNX Runtime supports mobile inference on Android [2]. CPU execution is selected to reduce the number of initial compatibility profiles. GPU, NPU, quantized, and LiteRT profiles can be added only after independent correctness and economics tests. LiteRT is an available Android inference option, not a guarantee that every accelerator supports every model [3].

Property	Initial proposed limit
Input format	UTF-8 text records with unique customer record IDs
Text length	Maximum 256 model tokens per record, including required special tokens
Task size	Up to 64 records, reduced for slower devices
Task payload	Maximum 256 KiB before transport compression
Output	One normalized 384-dimensional FP32 vector per accepted record
Task duration target	5 to 30 seconds after the model is loaded
Concurrent tasks	One task per device
Model package target	At most 200 MiB per approved download package
Active model memory gate	Measured peak resident app memory at most 768 MiB for this profile

Long text is rejected unless the customer explicitly selects a documented splitting or truncation policy before accepting a quote. Silent truncation is not permitted. Output identifiers preserve the customer's original ordering even when tasks finish out of order. For 64 records, the raw vectors occupy 98,304 bytes before metadata or encoding.

4.2 Workload admission rules

Each new profile must publish an exact model revision, license, runtime build, preprocessing rules, output schema, validation thresholds, resource limits, benchmark suite, and price unit. Customer-uploaded native code, arbitrary ONNX graphs, custom operators, scripts, and unrestricted model uploads are excluded from the initial service.

Future profiles are evaluated independently. Classification requires approved labels and a fixed scoring method. OCR requires document-level quality criteria. Transcription requires language, audio, and error-rate criteria. Generative tasks require stronger evaluation than checking whether two text outputs match and are not part of the initial paid catalog.

5 Android application and execution

5.1 Session model

The first release provides an explicit Start Session action and a visible compute screen. It defaults to a 30-minute session, which the user can shorten or extend up to two hours. The app stops acquiring tasks when the session expires. App-visible execution is the baseline; screen-off or background execution is enabled only for device and distribution combinations that pass a separate review.

If a foreground service is used, it must have a justified service type, required declarations, a visible notification, and a Stop action. Android restricts background foreground-service starts, and specialUse is subject to review [4, 5]. A notification alone is not approval to run indefinitely. The implementation must not mislabel general AI processing as media playback or device connectivity.

WorkManager handles eligible bounded scheduling, retry, and housekeeping. It is not the mechanism for promising unlimited unattended compute; Android documents quota constraints for long-running workers [6]. Automatic charging sessions and idle schedules are later features, conditional on policy acceptance and actual device behavior. No session silently restarts after a reboot or force-stop.

5.2 Default resource controls

Control	Proposed default
Power	External power required
Battery-equipped devices	Battery at least 80 percent to start; pause below 70 percent
Network	Unmetered connection required; no mobile data by default
Parallelism	One inference task; at most two CPU inference threads
Data budget	1 GiB per day including model downloads; owner may set a lower cap
Model cache	512 MiB maximum; explicit confirmation before larger use
Thermal policy	Reduce load at moderate status; cancel and pause at severe or worse
Stop behavior	Stop new leases immediately; request active inference cancellation
Session end	Release compute resources and return unfinished work

The app checks power, connectivity, storage, session status, and memory before taking a lease. It listens for thermal changes and rechecks constraints between records. Headroom polling, where supported, is no more frequent than once every ten seconds, following Android guidance [7]. Battery temperature is an additional signal, not a universal measurement of processor temperature.

After a thermal pause, the app waits at least five minutes and requires a stable acceptable state before resuming within an authorized session. If thermal reporting is missing or unreliable, sustained participation is disabled until that device profile passes a conservative validation process. Batteryless equipment uses its tested external-power policy instead of a fictitious battery percentage.

Pause responsiveness is measured during qualification. Profiles whose inference cannot be interrupted promptly must use smaller tasks or be excluded. The target is to stop resource-intensive

activity within five seconds of an explicit stop or severe thermal signal; that is a release requirement, not a guarantee provided by Android.

Owners can restrict sessions, reduce data and storage budgets, remove cached models, and disconnect a device. The interface displays actual task availability and can show Waiting for work. It does not manufacture jobs or show projected daily income when there is no paid demand.

6 System architecture

The reference stack uses a Kotlin Android client, a Go coordinator API, PostgreSQL for authoritative job and accounting records, an object store for encrypted payloads and results, and Python verification workers running a pinned model runtime. Redis may cache availability and rate limits, but it is not the authority for balances or job completion.

Component	Main records or function	Failure response
Customer API	Quotes, orders, keys, authorized webhooks	Retry with idempotency keys
Device gateway	Registration, heartbeats, leases	Stop new leases when unhealthy
Scheduler	Eligible profile and device matching	Recover from transactional job state
Model registry	Signed manifests and approved artifacts	Reject untrusted or revoked profiles
Verifier	Reference outputs and validation decisions	Hold results; do not assume acceptance
Accounting service	Double-entry journals and reconciliation	Freeze settlement on imbalance
Settlement service	Approved payouts and chain monitoring	Retry safely by settlement identifier
Customer dashboard	Progress, results, costs, receipts	Reflect durable API state

The coordinator uses database transactions for lease acquisition and logical task completion. A transactional outbox publishes notifications and queue events after state commits. Workers may receive messages more than once; each handler checks the authoritative record before acting. Exactly-once economic effects are enforced by unique identifiers and transactions, not assumed from queue delivery.

Task payloads use short-lived, task-scoped download permissions. The node contacts only approved AndroNode endpoints and approved model artifact origins. A task contains data and fixed execution parameters, never a shell command or an arbitrary customer URL to fetch. Results are size-limited and parsed as untrusted input.

AI computation remains offchain. Blockchain operations are limited to deposits, settlements, refunds, and withdrawals. Model weights, input text, result vectors, and personal device metadata are not written to a public blockchain.

7 Model registry and runtime profiles

A profile identifier binds the model artifact digest, upstream revision, tokenizer digest, pooling and normalization code revision, runtime version, execution backend, numeric precision, output schema, input limits, and validation policy. Changing any bound element creates a new profile identifier.

The profile also identifies the trusted verifier runtime build separately from the Android binary. Both use the same model semantics and are calibrated against the same fixtures; cross-platform binaries are not assumed identical. The release manifest is signed by a dedicated model-release key. The Android app verifies both the signature and SHA-256 file digests before loading a package. Runtime libraries ship through a signed app release; they are not delivered as executable task payloads. Downloaded models are approved data artifacts with bounded operators and shapes.

The registry retains license notices and records whether commercial use and redistribution are permitted. A proposed model is not activated until its license, conversion, mobile compatibility, and output quality are reviewed. Customers select a profile explicitly and can download its public specification and test vectors.

Each backend is a separate profile. An accelerator becoming unavailable must not silently change a job's precision or model. The scheduler may offer a different approved profile only with customer consent and a new quote when output semantics or pricing change.

Revocation stops new leases immediately. Outstanding tasks are either verified under the original safe profile or cancelled and reissued under a customer-approved replacement. Accepted contributor pay terms remain attached to the original lease.

8 Job lifecycle and scheduling

8.1 Customer orders and node tasks

A customer order contains many records. The coordinator groups them into short tasks for devices. Each task has a logical task ID; every execution attempt has its own lease ID. An order can succeed partially without losing accepted records.

State	Meaning	Next action
QUOTED	Price, profile, limits, and expiry presented	Customer accepts and funds
FUNDED	Finalized funding covers the maximum charge	Validate and queue records
QUEUED	Task awaits an eligible device	Atomically issue a lease
LEASED	One attempt has a fixed reward and deadline	Device executes or returns lease
SUBMITTED	Result uploaded and signed	Verify signature, schema, and output
VERIFYING	Trusted recomputation or review in progress	Accept or reject the attempt
ACCEPTED	Result passes profile rules	Create a provisional payment entry

State	Meaning	Next action
SETTLED	Payment credited onchain after review	Contributor may withdraw
EXPIRED or REJECTED	Attempt failed or missed its lease	Retry within order limits
CANCELLED or FAILED	No further execution authorized	Refund unspent customer funds

The table combines order and task states for readability; the implementation stores separate state machines. For example, a funded order may contain accepted, queued, and failed tasks at the same time. It becomes completed when all records are terminal, or partially completed when its deadline ends with only some accepted.

8.2 Assignment rules

The scheduler first filters by approved profile, session authorization, resource limits, cached model, estimated completion time, reputation, and remaining customer budget. It then selects a device using measured performance, fairness, and controlled exploration of new devices. A node does not need to hold or stake a native token to receive work.

The initial lease lasts at most 120 seconds with a server timestamp and expires no later than the customer order's execution deadline. No lease is issued unless estimated execution plus a 30-second upload margin fits inside the remaining interval. Nodes send a heartbeat every 20 seconds during a task. Heartbeats do not extend the deadline automatically. Each device may hold one active lease. Payload microbatches shrink when measured throughput suggests a task would exceed the 30-second execution target.

An attempt is payable only if its complete submission reaches the coordinator before lease expiry and the result later passes verification. A database transaction checks the server deadline and current fencing generation, then changes LEASED to SUBMITTED. Lease expiry and reassignment use the same lock and generation check. A task already SUBMITTED or VERIFYING cannot be reassigned simply because its original execution lease later expires. Verifier backlog cannot invalidate an on-time submission. An accepted logical task cannot create a second customer charge or second primary payment. A duplicate submission with the same lease ID and output digest returns the original receipt; a conflicting digest is rejected.

Each task receives at most three automatic attempts before final failure or an optional operator-funded fallback. Platform-caused retries and reference verification are funded by the service charge. A trusted fallback may deliver a separately labelled no-charge result; it creates neither a customer compute charge nor a fictitious device payout. Only accepted contributor work enters the paid 75/25 allocation.

8.3 Cancellation and partial completion

Customer cancellation stops new leases. Work already leased remains payable if submitted within its existing deadline and accepted. The cancellation response shows that limited outstanding liability. At the execution deadline, unfinished attempts stop and accepted records remain deliverable. Complete on-time submissions still in verification or review keep their maximum charges reserved until resolved or until the escrow hard close. They can be accepted after the execution deadline; the deadline limits submission, not asynchronous verification. Only accepted contributor records are charged.

The accepted contributor reward is fixed when the lease is issued. A customer cannot erase that obligation by cancelling after work is accepted. The operator's prefunding and settlement procedures must cover this rule.

9 Result verification

9.1 Launch validation

Every submitted output in the initial paid release is recomputed by a trusted verifier using the approved model profile. This deliberately duplicates inference. It provides a clear acceptance rule, but can make the service more expensive than simply running the job centrally. The pilot must measure that overhead rather than assume decentralization reduces cost.

For the reference embedding profile, the verifier checks record IDs, output count, vector dimensions, finite values, input and model digests, and normalization. It compares every device vector with a reference vector. The proposed FP32 launch thresholds require cosine similarity of at least 0.9999, maximum absolute component error no greater than 0.001, and vector norm between 0.999 and 1.001. These are candidate tolerances; the profile cannot be activated until calibration demonstrates that honest supported devices pass and deliberately invalid outputs fail.

All checks must pass. A zero vector, NaN, wrong record, mismatched model, or duplicated output inconsistent with its corresponding reference fails. Duplicate inputs may legitimately produce identical vectors. Hashes bind an output to a receipt but cannot establish numeric correctness. Floating-point differences make exact cross-device output hashes unsuitable as the only validation rule.

Correct model execution does not guarantee that the model is useful for a customer's application. The customer evaluates semantic quality on a sample before purchasing a batch. Verification establishes conformance to the chosen profile, not universal accuracy.

9.2 Rejections and disputes

A rejected attempt receives a reason code such as INVALID_SIGNATURE, WRONG_PROFILE, SCHEMA_ERROR, NONFINITE_OUTPUT, NUMERIC_MISMATCH, or LEASE_EXPIRED. The contributor can request review within seven days. Profile or verifier defects trigger reprocessing and correction from the operator reserve; they must not silently reduce already settled balances.

Customer output disputes must be opened within 48 hours of result availability. During this period contributor entries remain provisional. A dispute is evaluated against the accepted profile and quote, not a changed customer preference. Operator-caused quality failures are covered by refund or rerun. The operator initially adjudicates disputes, so this process is not trustless arbitration.

9.3 Later verification models

Sampled recomputation, independent replication, and hidden known-answer tasks are research extensions. None may replace full verification in the paid baseline without a new validation policy, measured adversarial testing, and explicit customer opt-in.

If a fraction a of tasks is randomly audited, an attacker submitting k independent invalid tasks is detected with probability 1 minus (1 minus a) raised to k, assuming audits are unpredictable and detect those invalid outputs. For example, a 10 percent audit rate detects one invalid task with only 10 percent probability. This does not certify each output, prevent selective cheating, or solve collusion.

Device attestation, a signed receipt, or a blockchain transaction is not a proof of AI execution. A compromised node could outsource computation elsewhere. Initial payments reward useful accepted outputs; they do not certify that all computation physically occurred on Android hardware.

10 Contributor identity and abuse prevention

Each device generates a signing key, preferably hardware-backed where supported. Android Keystore can protect non-exportable keys, with protection varying by hardware and configuration [8]. This key signs registration challenges, lease acceptance, and result envelopes. It is distinct from the owner's withdrawal wallet key and cannot authorize a withdrawal by itself.

The coordinator binds challenges to an account, device key, session, nonce, and short expiration. Signed results bind the job ID, task ID, lease ID, profile ID, input digest, output digest, and server-issued deadline. Replayed, expired, or profile-mismatched messages are rejected.

Play Integrity signals can help assess supported Play-distributed devices. Google recommends using them with other abuse controls [9]. These signals do not prove unique ownership, correct results, local inference, or customer-data confidentiality. Directly distributed or non-GMS installations have separate admission rules and cannot be presented as equivalently attested.

New devices begin in probation with low-value tasks and capped outstanding rewards. The proposed initial cap is $1 of provisional rewards per new device until it completes 100 accepted tasks and seven calendar days of clean activity. Accounts face an initial $10 provisional cap. Higher limits require measured reliability and risk review.

The operator monitors account clusters, replay patterns, output duplication, task completion anomalies, and abnormal registration rates. Shared IP addresses are not automatically treated as fraud because legitimate households and institutions may share a connection. Reputation affects allocation and risk limits; it does not multiply the payment for identical work.

An account can register multiple legitimate devices. Sybil resistance therefore focuses on preventing unearned payments, free promotional rewards, and concentration of unsettled exposure. It does not attempt to enforce one person per device. No signup emission or multi-level referral payment is included in the baseline.

11 Customer data and privacy

Ordinary inference requires a node to access its assigned inputs and model. A device owner or compromised device may inspect, copy, or retain that data. Transport encryption and Android app isolation do not create confidential computing against the node owner.

The initial service accepts only public, synthetic, licensed, or explicitly cleared non-sensitive data suitable for distribution to third-party devices. Personal records, credentials, private customer conversations, unreleased documents, regulated records, biometric data, and confidential model weights are excluded. Public availability alone does not remove copyright or personal-data restrictions.

Customers receive this boundary before upload and attest that they have rights to process and distribute the inputs. Automated screening is a supplementary control, not a guarantee that sensitive

content has been removed. If an order is not suitable, it is rejected before device assignment. Trusted-host processing, if offered later, must be a separately identified product.

Payloads are encrypted in transit and in operator storage. Signed task URLs expire after ten minutes and are scoped to one task. The app uses private storage, removes plaintext task files after completion or expiry, and avoids writing inputs to diagnostics. Deletion on an untrusted contributor's device cannot be guaranteed.

The proposed operator retention schedule is seven days after order completion for inputs and result artifacts, 30 days for technical dispute evidence, and 90 days for minimized security telemetry. A dispute can extend relevant evidence retention to 30 days after resolution. Financial records follow the applicable published recordkeeping schedule. Customers can request earlier deletion, subject to settlement disputes and legal retention duties.

Public receipts contain opaque IDs and hashes of manifests built with fresh random nonces. Raw input hashes can leak predictable text through guessing and are not published alone. Even salted commitments are metadata, not a substitute for protecting the underlying data. Wallet addresses and settlement events on a public chain remain publicly observable.

12 Pricing and contributor payments

12.1 Price formation

Customers pay a quoted amount per 1,000 accepted records for a specific profile, length band, validation policy, deadline, and data route. The quote includes verification, expected retries, transport, and the platform service charge. It expires after 15 minutes unless accepted and funded. No model-specific price is advertised as a market rate until measurements support it.

The proposed standard allocation is 75 percent of accepted compute charges to primary contributors and 25 percent to the operator service pool. The service pool funds trusted verification, task transport, storage, chain settlement costs, support, fraud losses, and operations. It is not all profit. There is no separate verifier reward added to the customer invoice after acceptance.

A lease states the exact contributor amount in the settlement asset's smallest units. Different profiles or length bands may quote different rates. The same work under the same quote has the same base reward regardless of token holdings. For paid version 1, each quoted per-record customer charge must be an integer number of asset units divisible by four. Each record therefore allocates exactly three quarters to its contributor and one quarter to the service pool, with no rounding dust. Unsupported price precision is rejected at quote creation.

Unfunded orders are not scheduled. When paid demand is absent, devices wait without accruing base rewards. Any grant-funded demonstration work is marked as sponsored and uses a separately funded budget. Sponsored volume cannot be reported as organic customer revenue.

12.2 Illustrative order economics

The following is an accounting example, not a price recommendation or observed benchmark. Assume 100,000 accepted records priced at $0.20 per 1,000 records and a settlement asset valued at $1 for illustration.

Item	Amount	Treatment
Customer compute charge	$20.00	100 units of 1,000 accepted records
Contributor allocation	$15.00	75 percent of compute charge
Operator service pool	$5.00	25 percent of compute charge
Trusted verification	$1.40	Illustrative internal cost
Storage and transport	$0.60	Illustrative internal cost
Retries and fraud allowance	$0.50	Illustrative internal cost
Settlement and RPC costs	$0.20	Illustrative internal cost
Variable operations and support	$0.80	Illustrative internal cost
Remaining contribution margin	$1.50	Before fixed salaries, development, tax, and other overhead

These example costs total $3.50 inside the $5 service pool. If trusted verification instead costs $4, the service cost becomes $6.10 and the order loses $1.10 before fixed overhead. Pricing must change or the profile must remain unavailable commercially. A large device count does not repair negative unit economics.

A device completing 1,000 accepted records under this example earns $0.15 gross. This says nothing about how long it takes or how many orders exist. If that device uses an incremental 5 watts for two hours at $0.15 per kWh, electricity costs $0.0015; hardware wear, cooling, connectivity, withdrawal fees, and taxes remain additional costs. Both power and electricity price here are hypothetical and must be replaced with measured local values.

12.3 Earnings presentation

The dashboard separates provisional rewards, onchain claimable balances, and withdrawn funds. It shows the profile, accepted units, exact rate, rejection reasons, and settlement transaction. Estimates use the owner's measured throughput and actual recent task availability, with a visible time window. No fixed daily return, guaranteed utilization, or hardware payback period is promised.

13 Settlement and customer funds

13.1 Settlement asset and chain

The intended paid release uses USDC on one selected low-fee EVM chain, subject to asset availability, jurisdictional review, and contract validation. The chain ID, official asset address, decimals, confirmation policy, escrow address, and administrator keys must be published before accepting deposits. No chain or token contract is claimed deployed by this specification.

The application accepts only the allowlisted asset on that chain. It does not automatically bridge deposits or accept lookalike tokens. The initial asset assumption is a six-decimal token; the deployment must explicitly verify the asset behavior. A different precision requires corresponding

accounting and test changes. Dollar displays are informational; balances and obligations are denominated in asset units, which may diverge from dollars.

13.2 Escrow operations

A customer funds an order escrow with a maximum charge, profile and quote commitment, payout allocation, and hard close timestamp. The proposed execution deadline is at most seven days after funding, with a hard close 14 days after funding. This budgets up to 24 hours for verification, 48 hours for customer review, the next daily settlement, and a recovery buffer. The settlement service targets closure by day 11. Longer orders require a separate versioned policy.

The contract supports fundOrder, settleChunk, closeOrder, refundRemainder, and withdraw functions. It stores deposited funds, cumulative allocated charges, contributor and operator claimable balances, processed settlement identifiers, and an order close timestamp. Each settlement chunk is limited to 100 recipient entries to bound transaction cost.

Settlement requires the published 2-of-3 administrative authorization. Each chunk commits to an order ID, unique settlement ID, receipt manifest hash, accepted charge, and recipient allocation. The contract checks that the order exists, the settlement is not replayed, cumulative charges do not exceed funding, amounts balance, and allocations obey the order's fixed 75/25 rule. It does not evaluate the AI result or decide whether an accepted charge is honest.

Every chunk charge is a multiple of four smallest asset units, matching its constituent record charges. Exactly three quarters becomes contributor credit and one quarter becomes operator credit. Recipient amounts must sum to the contributor total and match the fixed task rewards in the manifest; the contract enforces the aggregate ratio, while task-level correspondence remains an administrator responsibility. Cumulative allocations cannot exceed the order's quoted limits. Final closing requires reconciliation of every accepted task to exactly one settlement entry.

Claimable funds use a pull-payment design: a recipient withdraws its own credited balance to an authorized wallet. One failing recipient does not block other payouts. Reentrancy protection, supported token-transfer handling, amount checks, and chain-bound unique identifiers are required. Successful withdrawal changes the credited balance before the external token transfer.

13.3 Timing and refunds

Accepted tasks enter a provisional ledger. Undisputed entries become eligible for the next daily settlement after the 48-hour customer review period. This is a proposed operating schedule, not instant finality. Open disputes stop settlement for the affected entries. Operator-caused refunds after funds have settled are paid from operator reserves, not by clawing back unrelated contributors.

When an order closes, the customer receives unallocated escrow funds. If administrators stop operating, the customer can recover the remaining unallocated balance after the hard close timestamp without their signature. Before that timestamp, only explicitly finalized unspent funds can be refunded. Contributors with already credited balances can withdraw without the coordinator. Contributors whose work is only in the offchain provisional ledger still depend on administrators to authorize payment.

The interface makes this distinction explicit. The first contract has no cryptographic proof of uncredited work and no automatic way to reconstruct it from a device claim. If unresolved disputes or an outage prevent settlement before hard close, remaining unallocated funds become refundable and any valid

unpaid contributor obligation becomes an operator-reserve liability. It is not silently cancelled. No allocation can occur at or after hard close.

Network-wide caps of 500 settlement asset units in outstanding customer escrow and 50 units in unsettled contributor exposure apply to the paid pilot, backed by a separate 100-unit operator reserve. Unsettled exposure includes the maximum promised rewards of active leases as well as submitted and accepted unpaid work. New leases stop before either cap could be exceeded. If an operator-funded liability consumes reserves, new paid work remains paused until coverage is restored. These amounts have the stated dollar equivalents only under the illustrative one-dollar asset assumption.

13.4 Administration and accounting

Initial escrow contracts should be non-upgradeable. New versions use new deployments; migration requires an explicit user action. Two of three separate administrator keys authorize settlements and can pause new funding and new settlement allocations. Existing credited withdrawals and time-based recovery of unallocated customer funds remain available during a pause. Settlement already credited cannot be reversed administratively.

This design limits certain failure modes but does not prevent dishonest administrators from misallocating still-unallocated escrow within contract limits. An independent contract audit and adversarial tests are required before external funds. A public event record is an audit trail, not proof that administrators acted correctly.

The accounting service uses integer asset units and a double-entry journal. It reconciles finalized deposits, locked order funds, provisional obligations, credited claims, refunds, withdrawals, and operator revenue daily. Every economic event has one stable idempotency key. A reconciliation mismatch stops settlement and new funded jobs until investigated. Chain reorganizations reverse provisional chain observations before they can fund new work.

13.5 Withdrawal experience

Contributors link an external wallet through a domain-bound signed challenge. The node signing key is not a withdrawal key. Withdrawal-address changes require account reauthentication, confirmation from the existing wallet or a documented recovery process, and a 24-hour payout hold.

The dashboard proposes a $1 equivalent batching threshold to reduce fee friction, but contract-credited balances do not expire and have no protocol minimum for self-funded withdrawal. A relay may sponsor withdrawals only within a published budget. If users pay network fees themselves, the interface shows the fee and required native gas before confirmation. No fiat cash-out, exchange liquidity, or offchain redemption service is implied.

14 Native token policy

The base service does not require an AndroNode native token. Customer-funded stablecoin payments provide a direct unit of account for useful work, subject to distribution and operating approval. Test points in the engineering pilot are non-transferable, have no promised conversion value, and are not a commitment to a future airdrop.

A future native asset would require a separate approved token specification covering its necessity, ticker, supply, allocations, vesting, emissions budget, rights, governance limits, jurisdictional

requirements, and integration with actual customer demand. None of those parameters has been approved here. The protocol does not assume a token sale, token-price appreciation, buybacks, or revenue distributions.

Any future token must not be required merely to unlock ordinary contributor earnings or buy an artificial reward multiplier. Device work must not be converted into proof-of-work cryptocurrency mining and presented as AI activity. Token emissions, if ever adopted, must be disclosed separately from customer-funded payments and cannot be counted as operating revenue.

15 Product interfaces and developer API

15.1 Contributor application

The application provides device qualification, a resource settings screen, session controls, current task status, reward history, rejection details, and withdrawal links. Status labels include Not compatible, Ready, Waiting for work, Working, Paused by owner, Paused for temperature, Verifying, Provisional reward, Claimable, and Withdrawn.

The app reports whether a session requires remaining visible and whether background operation is enabled on that device. TV interfaces require remote-friendly navigation and an immediately accessible Stop control. The operator cannot remotely override owner limits to keep a device earning.

The client requests only permissions necessary for network access, approved notifications or services, and local app storage. It does not request contacts, call logs, location, personal photos, microphone, or camera access to process remotely supplied data. More intrusive future features require separate scope and consent.

15.2 Customer workflow

The customer selects a profile, reviews sample quality, uploads or registers an approved dataset, receives a quote, funds the order, and tracks accepted records. Results are retrieved through authenticated download URLs or a signed webhook notification. Exports contain customer record IDs, output vectors, profile revision, acceptance status, and receipt identifiers.

The initial product offers an asynchronous batch API with best-effort completion inside the accepted deadline. It does not promise guaranteed real-time latency. If expected capacity cannot meet the quote, the coordinator refuses the order or offers a later deadline before funding.

Endpoint	Purpose	Required protection
GET /v1/profiles	List approved profiles and limits	Public rate limits
POST /v1/quotes	Price a validated workload description	Customer API key and request limits
POST /v1/orders	Accept a quote and bind a funding reference	Idempotency key and ownership check
GET /v1/orders/{id}	Retrieve progress and charge totals	Customer ownership check
POST /v1/orders/{id}/cancel	Stop further task assignment	Idempotency key

Endpoint	Purpose	Required protection
GET /v1/orders/{id}/results	Retrieve an authenticated result manifest	Scoped download authorization
POST /v1/devices/register	Bind a device public key	Account authorization and nonce
POST /v1/leases/acquire	Request one eligible task	Active session and signed challenge
POST /v1/leases/{id}/result	Submit a signed output manifest	Lease binding and idempotency
GET /v1/rewards	View provisional and credited balances	Account authorization

API keys are stored as verifiers rather than plaintext and have per-project scopes. The quote fixes record counts, permitted length bands, maximum charge, and profile. Order ingestion rejects data that exceeds those limits instead of silently increasing the bill.

Webhooks include an event ID, timestamp, payload signature, and retry count. Customers deduplicate by event ID and can fetch authoritative order state. Webhook delivery failure cannot create a second order charge or discard already accepted results.

16 Security and operational recovery

16.1 Trust boundaries

The threat model includes malicious contributors, compromised customer accounts, hostile input files, replayed messages, dishonest administrators, model supply-chain attacks, and unavailable infrastructure. It also includes honest device interruption and numeric incompatibility, which must not be treated automatically as intentional fraud.

Failure or attack	Primary control	Remaining limit
Fabricated vectors	Full trusted recomputation	Verifier bugs or malicious operator
Replayed accepted result	Unique lease, task, and settlement IDs	Identifier implementation defects
Compromised node	Restricted workload and minimal permissions	Owner can inspect assigned plaintext
Malicious model package	Approved registry, signatures, parser limits	Runtime and supply-chain vulnerabilities
Coordinator outage	Durable state, expiring leases, recovery procedures	New tasks and provisional settlement stop
Administrator compromise	Separate keys, multisignature, exposure limits	Authorized collusion can misallocate funds
Chain outage or reorganization	Finality policy and paused new orders	Delayed funding and withdrawals

Failure or attack	Primary control	Remaining limit
Lost contributor device	Account recovery and separate wallet key	Unsubmitted work cannot be proven automatically

The verifier runs isolated workers with no unnecessary outbound network access, bounded resources, and read-only runtime images. The system scans input structure and tensor sizes before inference. Dependency versions are pinned and tracked in a software bill of materials. Emergency profile revocation and signed client updates address discovered vulnerabilities.

16.2 Recovery procedures

If the coordinator fails, devices stop taking work when leases cannot be acquired. Previously issued tasks can upload until expiry if the gateway is available. On recovery, the coordinator reconstructs active attempts from PostgreSQL and accepts no task twice. The accounting journal is replayed and reconciled before settlement resumes.

If the verifier fails, timely submissions remain in VERIFYING. The proposed operating target is to clear ordinary verification within 24 hours. Beyond that interval, new work pauses and affected users receive a status update. A system-caused backlog does not convert a timely valid submission into an expired lease.

If a model defect is discovered, the profile is suspended, affected orders are identified by profile ID, and results are rerun or refunded under the incident policy. Contributor compensation for operator-caused errors is funded from reserves. If reserves are insufficient, this is disclosed as a payment incident rather than hidden as normal verification rejection.

Operational targets for the pilot are database recovery point of at most five minutes and service recovery within four hours, backed by encrypted backups and restore drills. These are proposed targets until measured. Signing-key recovery and chain transactions require separate drills; restoring a database must not replay withdrawals.

17 Distribution and operating requirements

Google Play prohibits apps that mine cryptocurrency on devices and requires relevant declarations when users earn tokenized digital assets [10]. AndroNode is designed to execute customer-funded AI inference, not blockchain mining. That distinction does not establish automatic Play approval. The actual application, earning flow, wallet links, service behavior, and supported regions must pass review before a Play launch.

The engineering pilot can use an official signed APK with published checksums and controlled enrollment. Direct distribution still follows Android execution restrictions, user-consent requirements, and applicable law. It is not a bypass for prohibited behavior or a substitute for trustworthy app signing and updates.

The paid service launches only after the operator establishes its legal entity, terms, privacy policy, contributor eligibility, payment responsibilities, and supported jurisdictions. Stablecoin payments, wallet functionality, financial declarations, taxation, sanctions screening where applicable, and data-processing obligations are reviewed for the actual launch regions. No universal regulatory approval is claimed.

The first release excludes minors from paid participation and requires contributors to own the hardware or hold administrator authorization. Corporate, school, leased, and shared equipment requires documented permission. Android and Google names describe compatibility only; the protocol does not claim endorsement by Google or any device manufacturer.

18 Pilot and commercial launch plan

The roadmap uses evidence gates rather than guaranteed calendar dates. A feature advances only after the preceding release passes its technical, economic, and distribution criteria.

18.1 Engineering prototype

Build the Android client, one embedding profile, coordinator, reference verifier, and auditable test ledger. Use 20 supervised devices across at least five phone or tablet models. Process only synthetic or approved public data. Use test points and testnet settlement; collect no external customer deposits.

Exit requires correct stop behavior, thermal handling, device qualification, reproducible model outputs, duplicate-submission protection, and successful recovery from killed processes and network loss. Publish a benchmark method with device model, OS build, runtime, model hashes, session length, measured power, and error counts.

18.2 Closed network pilot

Expand to approximately 100 approved devices and at least ten tested models. Run one million reference-checked records across at least seven days of intermittent sessions. Include cold starts, shared Wi-Fi, memory pressure, device sleep, malicious output attempts, and coordinator or verifier outages. Grants fund any incentives and are reported as sponsored activity.

Pilot gate	Proposed pass criterion
Correctness	At least 99.9 percent of honest submissions pass calibrated conformance rules; investigate every systematic mismatch
Invalid-output screening	All deliberately seeded malformed outputs and tested corruptions rejected; publish test coverage
Task reliability	At least 95 percent of admitted leases submitted on time under declared test conditions
Retry-adjusted completion	At least 99 percent of eligible records complete within the quoted batch deadline
Owner control	95th-percentile stop latency at most five seconds; no model above the tested maximum
Resource enforcement	All simulated power, thermal, storage, and data-limit events trigger the specified pause behavior
Accounting	No unexplained difference between journals, task receipts, and testnet credits
Economics	Measured full cost per accepted unit, including 100 percent recomputation, documented for every offered profile

No finite test suite proves the absence of fraud or defects. The seeded-attack criterion describes coverage of tested cases, not a 100 percent real-world detection guarantee.

18.3 Limited paid beta

Before external funds, finish contract audit and fixes, publish deployment parameters, complete distribution and operating reviews, and rehearse refunds, disputes, withdrawals, and shutdown. Enroll at least three independent paying customers using non-sensitive workloads. Apply the 500-unit escrow and 50-unit unsettled contributor exposure caps until explicitly raised after review.

Commercial expansion requires four consecutive weeks of positive measured contribution margin at prices customers actually pay, excluding token emissions and grants. It also requires at least two repeat customers, no unresolved critical security findings, and evidence that demand and device availability can meet the offered deadlines.

If full recomputation makes the service commercially unattractive, the protocol remains a pilot while verification is redesigned or a different workload is tested. Reducing validation without disclosure or subsidizing permanent losses with speculative rewards is not a launch strategy.

18.4 Form factor and capacity expansion

Add Android TV boxes and additional Android equipment in separate supervised cohorts. Publish their sustained throughput, thermal behavior, energy use, and owner-control results. Support broader background operation only after appropriate implementation and policy approval. Add accelerators, new model profiles, and external verifiers individually rather than treating them as automatic consequences of adding devices.

19 Governance and protocol changes

The initial operator controls scheduling, model approvals, supported regions, and dispute decisions. A 2-of-3 administrator group controls onchain settlement permissions. Governance responsibilities and signers are published before the paid release. This is managed administration, not token-holder control.

Routine changes to pricing, reward shares, thresholds, or retention policies are published at least seven days before they apply to new orders. Existing quotes, accepted leases, and funded-order financial terms remain unchanged. Security incidents can pause new work immediately, with a public explanation as soon as it can be issued safely.

The operator publishes monthly figures for paid customer volume, sponsored volume, accepted units, rejection rates, verification cost, contributor payments, unpaid liabilities, operating reserves, and settlement incidents. Benchmarks distinguish registered devices from recently active devices and from devices that completed paid work. Claimed network capacity uses measured sustained throughput, not theoretical chip specifications.

Future decentralization requires demonstrated independent operators, standardized verifier behavior, portable receipts, and an adjudication process that does not depend entirely on one coordinator. These are implementation milestones. Issuing a governance token alone does not decentralize job validation or payment authorization.

20 Risks and limits

AndroNode may fail to attract sufficient paid workloads, and centralized compute may be cheaper after accounting for verification and data transfer. Contributors may receive little or no work. Gross rewards can be smaller than their operating costs, and purchasing hardware solely for projected earnings is unsupported by this specification.

Sustained inference consumes energy, produces heat, and may contribute to battery or hardware wear. Conservative settings reduce exposure but cannot eliminate it. Network-wide energy savings or environmental benefits require measured comparisons and are not assumed.

Customer inputs may be retained by malicious contributors. Model-conformant outputs can still be unsuitable for the customer's intended use. The baseline is not appropriate for confidential, safety-critical, or guaranteed real-time processing.

Administrator compromise, software defects, smart-contract errors, stablecoin depegging or issuer restrictions, chain outages, lost wallet keys, and policy changes can delay or destroy economic value. Audits reduce some risks but do not remove them. A contributor's offchain provisional balance has different protections from a contract-credited balance.

Device compatibility can change with OS, runtime, or manufacturer updates. A TV box that works during a short test may fail sustained operation. Published support lists and requalification are necessary operating functions, not optional maintenance.

21 Release specification

The following checklist defines the first paid protocol release. A checked item requires implementation evidence, not a marketing statement.

1. One approved embedding profile with model, tokenizer, runtime, license, artifact hashes, and public conformance fixtures.
2. An ARM64 Android client with qualification, explicit sessions, owner controls, signed updates, safe cancellation, and observable resource limits.
3. A transactional coordinator with leased tasks, idempotent submission, bounded retries, cancellation accounting, and capacity-aware quotes.
4. Full reference recomputation, documented numeric thresholds, rejection reasons, contributor appeals, and customer disputes.
5. A non-sensitive data policy with admission checks, scoped access, minimized logs, deletion procedures, and a clear node-owner threat boundary.
6. A double-entry ledger and audited, tested single-chain escrow with deposits, bounded settlement, pull withdrawals, refunds, and operator-failure recovery.
7. A customer API, contributor dashboard, public profile catalog, receipts, reconciliation reports, and incident communication process.
8. Distribution approval for the actual route, published launch regions, funded reserves, independent paying customers, and passed economic gates.

The release package must supply every deployment-specific value omitted by design here: exact model and runtime revisions, measured device list, validated tolerances, offered prices, selected chain and asset addresses, finality parameters, signer addresses, audited contract hashes, service terms, and support contacts. External funds are disabled until that package is complete.

22 Reference job and operating scenarios

22.1 Reference paid order

A customer submits 10,000 approved public English records, each within 256 tokens. The accepted quote uses the illustrative rate of $0.20 per 1,000 accepted records, so maximum compute charge is 2.000000 asset units. The contributor allocation is 1.500000 and the service allocation is 0.500000 if all records succeed. These are example values using a six-decimal settlement asset, not a published live price.

The coordinator creates 156 tasks of 64 records and one final task of 16 records, then adapts smaller tasks if required by eligible device speed. At the example rate, a 64-record task carries 0.012800 customer charge and a 0.009600 primary contributor reward. The last 16-record task carries 0.003200 charge and 0.002400 reward.

The device signs the lease, downloads only its assigned records, runs the pinned profile, and uploads output and a signed manifest. The reference verifier recomputes all vectors and records its comparison. Accepted results become downloadable. Contributor rewards remain provisional for the review window, then enter an onchain settlement chunk and become claimable.

If only 9,000 records are submitted on time and subsequently accepted, the final customer compute charge is 1.800000, the primary contributor allocation is 1.350000, the service allocation is 0.450000, and the unspent escrow refund is 0.200000. The output manifest identifies which 1,000 records were not completed. Refund calculation waits for resolution of timely pending submissions, subject to the hard-close recovery rule. Failed attempts do not create additional charges.

22.2 Required task envelope

Field	Binding or rule
protocol_version	Selects message and validation rules
order_id and task_id	Identify funded work and its logical record group
lease_id and device_key_id	Bind one attempt to one registered signer
profile_id	Commits model, tokenizer, runtime, backend, and thresholds
input_manifest_digest	Binds the exact ordered task input
record_count and limits	Bound record, token, memory, and output sizes
reward_asset and reward_units	Fix the contributor payment in integer units
issued_at and expires_at	Server-authoritative lease interval
nonce	Prevents reuse across challenges

Field	Binding or rule
coordinator_signature	Authenticates the offered lease

The result envelope adds the lease ID, output digest, completed record count, execution status, and device signature. Client-reported runtime, temperatures, and device information are diagnostic claims, not trusted billing evidence. A reference-validation receipt records the verifier profile, decision, reason code, comparison summary, and timestamp.

22.3 Recovery examples

If a node loses Wi-Fi before submission, it can retry the same upload within the existing lease deadline. After expiry, it discards the payable claim and the task can be reassigned. If an acknowledgment is lost after submission, resending the same lease and output digest returns the existing receipt instead of paying twice.

If the owner presses Stop, the app stops acquiring work and requests active inference cancellation. Unfinished work earns no partial reward unless a future profile explicitly prices record-level partial completion. A device is never penalized financially for protecting its temperature or obeying an owner stop request; repeated interruptions may reduce the jobs it can reliably receive.

If the chain is unavailable, onchain withdrawals and new funding can be delayed. New leases stop when settlement exposure caps are reached. The app displays the actual provisional balance rather than a fabricated successful payout. Once the chain recovers, finality and reconciliation checks run before settlement resumes.

23 Technical sources

Sources were checked on 2 November 2026. They support the named platform capabilities and restrictions, not the unimplemented AndroNode performance targets or commercial assumptions.

[1] Sentence Transformers. all-MiniLM-L6-v2 model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[2] ONNX Runtime. Get started with ONNX Runtime Mobile. https://onnxruntime.ai/docs/get-started/with-mobile.html

[3] Google AI Edge. LiteRT for Android. https://developers.google.com/edge/litert/android

[4] Android Developers. Restrictions on starting a foreground service from the background. https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start

[5] Android Developers and Google Play. Foreground service types and declaration requirements. https://developer.android.com/develop/background-work/services/fgs/service-types and https://support.google.com/googleplay/android-developer/answer/13392821?hl=en

[6] Android Developers. Support for long-running workers. https://developer.android.com/develop/background-work/background-tasks/persistent/how-to/long-running

[7] Android Developers. Thermal API. https://developer.android.com/games/optimize/adpf/thermal

[8] Android Developers. Android Keystore system. https://developer.android.com/privacy-and-security/keystore

[9] Android Developers. Overview of the Play Integrity API. https://developer.android.com/google/play/integrity/overview

[10] Google Play. Blockchain-based content policy. https://support.google.com/googleplay/android-developer/answer/13607354?hl=en